

Módulo 6 – Capítulo 06 – Sección 01

El problema de evaluar RAG: múltiples componentes y métricas en conflicto

Evaluar un sistema RAG es cualitativamente más difícil que evaluar cualquiera de sus componentes en aislamiento. Para evaluar un modelo de clasificación, existe una métrica clara (accuracy, F1, AUC) calculada sobre un dataset etiquetado. Para evaluar un LLM de propósito general, existen benchmarks estándar (MMLU, HumanEval, GSM8K). Pero un sistema RAG es un pipeline de al menos dos subsistemas —el retriever y el generador— que tienen métricas propias, parcialmente en conflicto, y cuya interacción produce la calidad observada por el usuario. Esta complejidad de evaluación no es una limitación técnica superable: es una propiedad estructural del sistema que requiere una estrategia de evaluación multi-nivel diseñada explícitamente para manejarla.

El **conflicto de métricas** entre el retriever y el generador es el primer problema. Un retriever que maximiza Recall@K recuperando 20 chunks en lugar de 5 mejora la probabilidad de que el chunk correcto esté en el contexto, pero puede introducir chunks irrelevantes que confunden al LLM generador —el fenómeno de "lost-in-the-middle" donde el modelo da menos atención a la información relevante cuando está rodeada de información irrelevante. Un generador configurado para ser conservador y responder solo cuando tiene alta confianza puede tener alta faithfulness (solo afirma lo que está en el contexto) pero baja answer relevancy (omite información que el usuario necesita porque el LLM no la considera suficientemente "segura"). Optimizar una sola métrica puede degradar las otras. Esta interdependencia exige una estrategia de evaluación que monitoree todas las métricas simultáneamente y defina umbrales mínimos para cada una en lugar de maximizar una única métrica compuesta.

El segundo problema es la **dificultad de diagnosticar el componente responsable** de una degradación observada. Cuando los usuarios reportan que el sistema "da respuestas incorrectas", la causa puede estar en el retriever (no encontró el chunk correcto), en el generador (ignoró el chunk correcto y respondió desde su conocimiento paramétrico), en el chunking (el chunk correcto fue cortado y la respuesta está distribuida entre dos chunks que

se recuperan por separado), o en el prompt template (las instrucciones al LLM no son suficientemente claras sobre cómo usar el contexto). Sin evaluación separada de cada componente, el diagnóstico es una búsqueda ciega.

El tercer problema es la **escala y el costo de la evaluación**. La evaluación humana de la calidad de las respuestas es la más confiable pero la menos escalable: evaluar 1000 respuestas requiere decenas de horas de trabajo experto. La evaluación automatizada con LLM-as-judge (usar GPT-4o o Claude como evaluador de calidad de las respuestas generadas) es escalable pero introduce sesgos del modelo evaluador: preferencia por respuestas verbosas, sensibilidad al orden de presentación de las respuestas, inconsistencia entre ejecuciones. La solución es una estrategia de evaluación en capas: métricas automáticas basadas en el dataset de evaluación para ciclos de iteración rápida, evaluación con LLM-as-judge para muestras del tráfico de producción, y evaluación humana periódica para calibrar la confiabilidad del LLM-as-judge.

Dimensiones del problema de evaluación

- **Desacoplamiento de métricas:** el retriever se evalúa con Recall@K sobre un dataset de qrels; el generador se evalúa con faithfulness y answer relevancy; estos datasets pueden construirse independientemente y deben hacerse en paralelo desde el inicio del proyecto.
- **Faithfulness vs. answer completeness:** un sistema que siempre responde "no tengo información suficiente" tiene faithfulness perfecta pero answer relevancy cero; el balance correcto requiere umbrales mínimos en todas las dimensiones simultáneamente, no optimización de una sola métrica.
- **LLM-as-judge con calibración humana:** usar GPT-4o o Claude Sonnet como evaluador automático de faithfulness y answer relevancy; validar la calibración del LLM-judge contra evaluación humana sobre un subconjunto de 50–100 respuestas para cuantificar el agreement; ajustar el prompt del judge si el agreement es bajo.
- **Golden dataset representativo:** el dataset de evaluación debe cubrir la distribución real de queries (factual, comparativo, multi-hop, temporal, ambiguo); un dataset sesgado hacia queries fáciles produce métricas infladas que no predicen la experiencia real del usuario.
- **Evaluación de componentes intermedios:** además del retriever y el generador, evaluar la calidad del chunking (¿los chunks tienen cohesión semántica?), de los metadatos (¿están completos y correctos?) y de los embeddings (¿vectores similares corresponden a contenidos semánticamente relacionados?).

- **Ciclo de evaluación continua:** ejecutar la evaluación automatizada en CI/CD para cada cambio del pipeline; la evaluación no es un evento único de lanzamiento sino un proceso continuo que detecta regresiones antes de que lleguen a producción.

Nota del Arquitecto: El error de evaluación más común es construir el golden dataset después de desarrollar el sistema, utilizando las queries que el sistema ya maneja bien. Esto produce un dataset sesgado que sobreestima la calidad del sistema y es incapaz de detectar sus puntos ciegos. El golden dataset correcto se construye antes de desarrollar el sistema, a partir de la distribución real de queries que los usuarios reales harán, incluyendo las más difíciles y las más ambiguas.

La complejidad del problema de evaluación de RAG no justifica la tentación de ignorarlo: la evaluación sistemática es precisamente lo que distingue un sistema RAG de producción confiable de uno que funciona en el demo pero degrada en producción. La siguiente sección presenta el framework RAGAS como la implementación más adoptada de esta estrategia de evaluación multi-dimensional.

Módulo 6 – Capítulo 06 – Sección 02

RAGAS: faithfulness, answer relevancy, context precision, context recall

RAGAS (Retrieval-Augmented Generation Assessment), disponible como librería Python open source en github.com/explodinggradients/ragas, es el framework de evaluación de sistemas RAG más adoptado en la industria. Su contribución principal es definir cuatro métricas core —dos sobre la generación y dos sobre la recuperación— que juntas cubren las dimensiones críticas de calidad del sistema completo, y proporcionar implementaciones ejecutables automáticamente sobre cualquier dataset de evaluación que el equipo construya. La arquitectura de evaluación de RAGAS usa un LLM como juez para calcular cada métrica, lo que permite la evaluación sin referencias de respuesta ground truth para algunas métricas, pero introduce una dependencia de la calidad del modelo evaluador.

Faithfulness mide qué fracción de las afirmaciones en la respuesta generada están directamente soportadas por el contexto recuperado. El algoritmo de RAGAS primero descompone la respuesta generada en declaraciones individuales usando el LLM evaluador ("La temperatura de ebullición del agua es 100°C a nivel del mar", "A mayor altitud, menor punto de ebullición"), luego verifica cada declaración contra el texto de los chunks recuperados para determinar si está o no soportada por evidencia explícita en el contexto. $\text{Faithfulness} = (\text{declaraciones soportadas por el contexto}) / (\text{total de declaraciones en la respuesta})$. Un score de faithfulness bajo (<0.70) indica que el sistema genera afirmaciones no fundamentadas en el contexto recuperado —hallucinations—, que pueden ser consecuencia de que el retriever no encontró la información relevante (el LLM la inventa) o de que el LLM ignora el contexto disponible y responde desde su conocimiento paramétrico.

Answer Relevancy mide qué tan directamente responde la respuesta generada a la pregunta del usuario, sin penalizar por información incorrecta (eso lo mide faithfulness) sino por información irrelevante, evasiva o incompleta. El algoritmo de RAGAS usa un enfoque ingenioso: genera con el LLM evaluador múltiples preguntas que podrían ser respondidas por la respuesta generada, y calcula la similitud coseno entre esas preguntas generadas y la pregunta original del usuario. Si las preguntas generadas son similares a la original, la

respuesta es relevante; si son distintas, la respuesta es evasiva o habla de algo diferente. Answer Relevancy penaliza respuestas que dicen "no puedo responder sobre eso" cuando la query tenía respuesta, respuestas que responden una pregunta diferente a la formulada, o respuestas que incluyen mucha información no solicitada.

Context Precision evalúa la calidad de la recuperación desde el lado de la precisión: ¿qué fracción de los chunks en el contexto recuperado son relevantes para responder la query? Esta métrica diagnostica el problema opuesto al de Recall@K: no si el chunk correcto está en el contexto, sino si los chunks incorrectos que también están en el contexto están contaminando el contexto del LLM. Un Context Precision bajo indica que el retriever recupera muchos chunks irrelevantes que el LLM debe ignorar —lo que incrementa el riesgo de que el "lost-in-the-middle" effect produzca respuestas que usen los chunks incorrectos.

Context Recall mide qué fracción de la información necesaria para responder la query está presente en el contexto recuperado. Esta métrica sí requiere una respuesta ground truth para comparar: RAGAS descompone la respuesta ground truth en declaraciones y verifica cuáles de esas declaraciones están soportadas por el contexto recuperado. $\text{Context Recall} = (\text{declaraciones del ground truth soportadas por el contexto}) / (\text{total de declaraciones del ground truth})$. Un Context Recall bajo indica que el retriever no recuperó los chunks con la información necesaria para responder correctamente.

La combinación de las cuatro métricas permite un diagnóstico preciso del componente responsable de una degradación de calidad: Context Recall bajo + faithfulness baja → problema del retriever (no recuperó la información correcta, el LLM la inventó); Context Recall alto + faithfulness baja → problema del generador (el contexto tiene la información pero el LLM la ignoró); Context Precision baja + faithfulness alta → el retriever recupera muchos chunks irrelevantes pero el LLM los ignora correctamente; Answer Relevancy baja → el LLM responde de forma evasiva o habla de otro tema.

Las cuatro métricas core de RAGAS

- **Faithfulness:** declaraciones en la respuesta soportadas por el contexto / total de declaraciones; rango [0,1]; <0.70 indica hallucinations sistemáticas; calculada sin necesidad de ground truth de respuesta.
- **Answer Relevancy:** similitud entre las preguntas que podría responder la respuesta generada y la pregunta original; rango [0,1]; penaliza respuestas evasivas o que hablan de otro tema; calculada sin necesidad de ground truth.

- **Context Precision:** fracción de los chunks del contexto que son relevantes para la query; rango [0,1]; diagnostica contaminación del contexto por chunks irrelevantes; requiere un LLM evaluador para clasificar cada chunk.
- **Context Recall:** fracción de la información del ground truth presente en el contexto recuperado; rango [0,1]; requiere respuesta ground truth; diagnostica directamente si el retriever encontró la información necesaria.
- **RAGAS Score:** media armónica de las cuatro métricas; número único para comparar configuraciones en experimentos; monitorear también las métricas individuales para diagnosticar degradaciones específicas.
- **Configuración práctica:** `pip install ragas` ; preparar dataset con columnas `question` , `answer` , `contexts[]` , `ground_truth` ; ejecutar `evaluate(dataset, metrics=[faithfulness, answer_relevancy, context_precision, context_recall])` ; usar GPT-4o o Claude Sonnet como evaluador LLM para mayor confiabilidad.

Nota del Arquitecto: RAGAS puede producir scores inconsistentes entre ejecuciones porque el LLM evaluador tiene variabilidad inherente. Para usar RAGAS como gate en CI/CD, ejecutar la evaluación tres veces y promediar, o usar `ragas.metrics.critique` con `temperatura=0` para reducir la variabilidad. Un delta de <0.03 en cualquier métrica entre dos ejecuciones es ruido normal; un delta >0.05 indica que el sistema cambió significativamente.

RAGAS proporciona el vocabulario cuantitativo para comunicar la calidad del sistema RAG de forma objetiva y descomponible. Con las cuatro métricas monitoreadas, la siguiente sección aborda cómo la evaluación separada del retriever proporciona el diagnóstico más eficiente de los problemas más frecuentes.

Módulo 6 – Capítulo 06 – Sección 03

Evaluación del retriever: separada de la evaluación del generador

La evaluación separada del retriever es probablemente la práctica de ingeniería de calidad más infrautilizada en proyectos RAG de producción, y también la más eficiente en términos de tiempo-resultado. Cuando la calidad del sistema degrada —cuando los usuarios reportan respuestas incorrectas o el equipo observa una caída en las métricas de RAGAS—, la primera pregunta que debe responderse antes de cualquier optimización es: ¿la degradación ocurrió en el retriever o en el generador? Responder esta pregunta sin evaluación separada del retriever requiere análisis manual de casos individuales, un proceso que puede tardar horas. Con el dataset de evaluación del retriever y las métricas de Recall@K actualizadas, la respuesta tarda minutos.

La evaluación del retriever en aislamiento requiere un tipo específico de dataset: pares `(query, [chunk_ids_relevantes])` anotados, donde los `chunk_ids` identifican unívocamente los chunks del índice vectorial que contienen la información necesaria para responder la query. Este dataset de retrieval se diferencia del dataset completo de RAGAS en un punto clave: no requiere ni la respuesta generada ni la respuesta ground truth, solo la identificación de los chunks relevantes en el índice. Con este dataset, se puede calcular Recall@K, MRR y NDCG ejecutando solo la búsqueda vectorial y comparando los resultados con los qrels, sin necesidad de ejecutar el LLM generador ni el evaluador de RAGAS. Esto hace la evaluación del retriever significativamente más rápida y más económica que la evaluación end-to-end.

La **construcción del dataset de evaluación del retriever** puede hacerse de tres formas con distintos trade-offs de calidad y costo. La anotación humana por expertos del dominio produce el dataset de mayor calidad: el experto formula queries representativas del uso real, identifica los chunks del índice que contienen la respuesta (qrels) y opcionalmente asigna scores de relevancia (0/1/2). El costo es de 5–15 minutos por query, lo que hace que 200 queries anotadas representen 3–5 días de trabajo experto. La generación sintética con LLM es más rápida: dado un chunk del índice, el LLM genera 3–5 preguntas que ese chunk

respondería; las preguntas generadas se usan como queries y el chunk original como el documento relevante (qrel). El riesgo de este enfoque es el sesgo de homogeneidad: las preguntas generadas por el LLM tienden a ser formuladas de forma similar al texto del chunk, lo que hace que el retriever las recupere con mayor facilidad que las queries reales de usuarios. La extracción de logs de producción con feedback de usuario (thumbs up/down, reformulaciones, ausencia de seguimiento) produce el dataset de mayor representatividad del uso real, pero requiere suficiente tráfico acumulado y un sistema de tracking implementado desde el primer día.

La evaluación del retriever debe segmentarse por **tipo de query** para identificar patrones de fallo específicos. Un Recall@5 global de 0.82 puede ocultar un Recall@5 de 0.40 para queries temporales (preguntas sobre la versión más reciente de una política, el estado actual de un proyecto) o un Recall@5 de 0.30 para queries multi-hop (que requieren combinar información de múltiples documentos). La segmentación revela el tipo de query donde el retriever falla sistemáticamente y permite enfocar las optimizaciones en ese tipo específico en lugar de aplicar cambios globales que pueden mejorar el recall en queries fáciles sin ayudar en las difíciles.

La evaluación del retriever también debe incluir **métricas de latencia** como dimensión de calidad: un retriever que tiene Recall@5 de 0.88 pero p99 de 3 segundos puede ser inaceptable para la experiencia de usuario objetivo aunque su recall sea excelente. Medir la distribución de latencias (p50, p95, p99) del retriever bajo carga realista es parte de la evaluación completa, no una consideración separada de infraestructura.

Aspectos técnicos de la evaluación del retriever

- **Dataset de retrieval con qrels:** formato estándar TREC donde cada par (query_id, doc_id) tiene un score de relevancia 0/1/2; permite calcular Recall@K, MRR y NDCG con relevance gradada; separado del dataset completo de RAGAS que requiere respuestas generadas.
- **Generación sintética con validación:** LLM genera preguntas desde chunks (técnica "generate questions from chunk") → LLM más potente valida la calidad de las preguntas generadas → dataset listo para evaluación; rápido pero con sesgo de homogeneidad; las preguntas sintéticas son sistemáticamente más fáciles de recuperar que las queries reales.
- **Hard negative mining para el dataset:** identificar chunks que son superficialmente similares al correcto pero no contienen la respuesta; incluirlos como qrels negativos

difíciles; un retriever que no distingue los negativos difíciles producirá hallucinations al enviar chunks semánticamente similares pero incorrectos al LLM.

- **Segmentación por tipo de query:** calcular Recall@K por categoría (factual_simple, comparativo, temporal, multi_hop, ambiguo); un recall global de 0.82 puede ocultar un recall de 0.40 en queries temporales que el retriever no maneja bien.
- **Evaluación de latencia como métrica:** medir p50, p95, p99 del tiempo de búsqueda vectorial en condiciones de carga realistas (concurrent queries); un retriever con recall excelente pero p99 inaceptable puede requerir optimizaciones de infraestructura antes de optimizaciones de calidad.
- **Drift detection en el retriever:** monitorear Recall@K en producción usando señales implícitas (reformulaciones de la misma query en <30 segundos, queries sin respuesta positiva) o evaluación asíncrona con LLM-judge; alertar cuando Recall@K estimado cae >5 puntos respecto al baseline.

La evaluación del retriever en aislamiento es la herramienta de diagnóstico más eficiente en el arsenal del AI Engineer: proporciona evidencia directa de si el componente más frecuentemente responsable de la degradación de calidad está funcionando correctamente o no. La siguiente sección aborda la construcción del golden dataset que hace posible esta evaluación con la calidad y representatividad necesarias.

Módulo 6 – Capítulo 06 – Sección 04

Construcción de datasets de evaluación: golden datasets y anotación humana

Un golden dataset de evaluación es el activo más valioso de la infraestructura de calidad de un sistema RAG. Es más valioso que el código del pipeline, que el índice vectorial o que el prompt template, porque sin él no hay forma de saber objetivamente si el sistema funciona bien o si los cambios que se hacen lo mejoran o lo degradan. Los equipos que operan sistemas RAG sin un golden dataset de evaluación están navegando sin instrumentos: pueden intuir que el sistema "funciona mejor" después de un cambio, pero no pueden cuantificar el delta ni compararlo con cambios anteriores. Los equipos que tienen un golden dataset de calidad convierten el desarrollo de RAG en un proceso de ingeniería medible donde cada experimento produce evidencia cuantitativa.

La **tipología de queries** que debe cubrir el golden dataset es el primer diseño que debe hacerse, antes de construir ninguna query específica. El dataset representativo debe incluir: queries factuales simples donde la respuesta está en un único chunk (el tipo más común en QA empresarial); queries que requieren síntesis de información de múltiples chunks del mismo dominio; queries comparativas que exigen recuperar información sobre dos o más entidades y compararlas; queries temporales que dependen de la fecha de los documentos o de la versión más reciente de una política; queries adversariales para las que no hay respuesta en el corpus (el sistema debe responder "no tengo información"); y queries ambiguas que pueden interpretarse de múltiples formas. La distribución proporcional aproximada que refleja el uso real en sistemas de asistentes empresariales es: 40% factual simple, 30% síntesis/comparativo, 15% temporal, 15% adversarial/ambiguo. Un dataset sesgado hacia queries fáciles produce métricas infladas que no predicen el comportamiento del sistema ante queries difíciles.

El método de **anotación humana por expertos del dominio** produce el dataset de mayor calidad pero requiere inversión de tiempo significativa. Un experto del dominio (alguien que conoce el corpus y puede identificar los chunks relevantes para cada query) tarda entre 5 y 15 minutos por query dependiendo de la complejidad: formular la query, buscar

manualmente en el corpus los chunks relevantes, asignar relevance scores (0/1/2), y escribir opcionalmente una respuesta ground truth. Un dataset de 200 queries con este método requiere entre 20 y 50 horas de trabajo experto, lo que es un costo significativo pero que se amortiza en cada ciclo de evaluación posterior. La calidad de la anotación humana determina la calidad de todas las métricas calculadas sobre el dataset: un qrel incorrecto produce métricas incorrectas que llevan a decisiones de optimización equivocadas.

La **generación sintética con LLM y validación** reduce el tiempo de construcción del dataset en un orden de magnitud. El proceso es: (1) para cada chunk del índice, usar Claude Haiku o GPT-4o-mini para generar 3–5 preguntas que el chunk respondería; (2) usar un LLM más potente (Claude Sonnet o GPT-4o) como juez para filtrar las preguntas que son demasiado similares al texto del chunk, demasiado específicas para ser queries reales, o con respuestas incorrectas en el ground truth generado; (3) muestrear uniformemente de las preguntas válidas para producir el dataset final con cobertura equilibrada de todos los tipos de chunks del corpus. El riesgo inherente de este método es el sesgo de facilidad: las preguntas generadas por el LLM tienden a usar el mismo vocabulario que el chunk, haciendo que el retriever las encuentre con mayor facilidad que las preguntas reales formuladas por usuarios humanos con su propio vocabulario.

El dataset debe incluir también un **subset adversarial**: queries para las que no existe respuesta en el corpus. Este subset es crítico para evaluar si el sistema rechaza correctamente las preguntas sin información disponible o si alucina respuestas. Preguntas sobre eventos posteriores a la fecha de corte del corpus, sobre temas fuera del dominio del sistema, o sobre entidades que no aparecen en el corpus son buenos candidatos para el subset adversarial. El sistema correcto responde "no encontré información relevante en el corpus" para estas queries; un sistema que las responde con confianza está alucinando.

Componentes de un golden dataset de evaluación

- **Query set representativo**: 100–500 queries que cubren la distribución real de uso; ratio aproximado 40% factual simple / 30% síntesis-comparativo / 15% temporal / 15% adversarial; incluir queries fáciles, medias y difíciles en proporciones similares.
- **Relevance judgments (qrels)**: para cada query, lista de chunk_ids con score 0/1/2; anotar chunks parcialmente relevantes (score=1) que dan información relacionada pero no completa; permite calcular NDCG con relevance gradada además de Recall binario.
- **Ground truth answers**: respuestas de referencia por expertos del dominio; permiten calcular métricas de generación (answer correctness, answer similarity) además de las

métricas del retriever; más costosas pero indispensables para evaluación end-to-end con RAGAS.

- **Subset adversarial:** queries para las que no hay respuesta en el corpus (eventos fuera del corpus, temas fuera del dominio, preguntas sobre entidades inexistentes); el sistema debe responder con abstención, no con alucinación; crítico para evaluar la calibración del sistema.
- **Versionado del golden dataset:** almacenar el dataset bajo control de versiones (Git LFS para archivos grandes); registrar qué versión del dataset se usó para medir cada versión del sistema; el historial de métricas solo es reproducible si el historial del dataset está preservado.
- **Pipeline de generación sintética con validación:** LLM pequeño para generar preguntas desde chunks → LLM potente para filtrar preguntas de baja calidad → muestreo estratificado para cobertura equilibrada de tipos de query; más rápido que anotación humana pero requiere validación posterior.

Nota del Arquitecto: El golden dataset debe construirse con las queries MÁS DIFÍCILES que el sistema debe manejar, no las más típicas. Las queries típicas fáciles se resuelven solas incluso con un sistema mediocre; las queries difíciles son las que revelan las limitaciones reales del sistema y las que los usuarios recordarán cuando el sistema falla. Un dataset de 200 queries donde el 80% son fáciles produce un Recall@5 de 0.90 que no predice el comportamiento ante las queries que importan.

Un golden dataset de alta calidad construido en la primera semana del proyecto es una inversión que retorna en cada ciclo de iteración posterior. Sin él, el desarrollo del sistema RAG es esencialmente experimentación no controlada. La siguiente sección aborda cómo usar este dataset en el monitoreo continuo de la calidad del sistema en producción.

Módulo 6 – Capítulo 06 – Sección 05

Evaluación continua en producción: detecting drift y degradaciones

La evaluación sobre el golden dataset durante el desarrollo responde a la pregunta "¿el sistema funciona bien bajo condiciones controladas?". La evaluación continua en producción responde a la pregunta más importante: "¿el sistema sigue funcionando bien a medida que el corpus, las queries de los usuarios y los modelos del pipeline evolucionan?". Los sistemas RAG se degradan silenciosamente en producción por múltiples causas que no son visibles en los datasets de evaluación estáticos: el corpus cambia con la incorporación de documentos sobre temas nuevos o con la eliminación de documentos clave; la distribución de queries cambia a medida que los usuarios descubren nuevos casos de uso o cuando el sistema se expande a nuevos departamentos; los modelos del pipeline se actualizan por deprecación del proveedor; y el índice vectorial se fragmenta gradualmente por las operaciones de actualización y eliminación. Detectar estas degradaciones antes de que los usuarios las reporten requiere un sistema de monitoreo de calidad específicamente diseñado para pipelines RAG.

El **tracing completo del pipeline** es el prerequisite de todo lo demás. Cada solicitud al sistema RAG debe instrumentarse con spans de OpenTelemetry que registren: el embedding de la query (latencia, modelo usado), la búsqueda vectorial (latencia, K, scores de similitud de los chunks recuperados, IDs de los chunks), el reranking si existe (latencia, scores finales), la construcción del contexto (número de tokens, chunks incluidos), la llamada al LLM (latencia, input tokens, output tokens, modelo), y la respuesta generada. Esta información almacenada en Langfuse, LangSmith, Phoenix de Arize AI o un sistema de trazas compatible con OpenTelemetry es la materia prima para todos los análisis de calidad en producción. Sin tracing, es imposible responder retrospectivamente "¿por qué el sistema dio esa respuesta a ese usuario en ese momento?".

La **evaluación asíncrona con LLM-as-judge** escala la evaluación de calidad a tráfico de producción sin impactar la latencia de las solicitudes. El mecanismo es sencillo: para un sample aleatorio del 1–5% de las solicitudes de producción, un proceso asíncrono (ejecutado

en un worker separado con delay de segundos a minutos) extrae la query, los chunks recuperados y la respuesta generada de las trazas, los envía al LLM evaluador (GPT-4o o Claude Sonnet) con los mismos prompts de faithfulness y answer relevancy que usa RAGAS, y almacena los scores en la base de datos de métricas de producción. Los scores diarios se agregan en dashboards (Grafana, Datadog, Langfuse UI) y se configura una alerta cuando cualquier métrica cae más de 10 puntos porcentuales respecto al baseline establecido en el dataset de evaluación offline.

El **embedding drift detection** monitorea cambios en la distribución de scores de similitud de las búsquedas diarias. Cuando el corpus es adecuado para las queries de los usuarios, los scores de similitud del top-1 chunk tienen una distribución característica (media alta, varianza baja). Si el corpus deja de cubrir los temas que los usuarios preguntan —por ejemplo, si el corpus es de documentación v3 de un producto pero los usuarios empiezan a preguntar sobre la v4 recién lanzada— la distribución de scores de similitud se desplaza hacia valores más bajos (los chunks más similares son mediocres porque el corpus no tiene la información relevante). Monitorear la distribución diaria de scores de similitud del top-1 chunk y alertar cuando la media cae más de 5 puntos respecto al baseline es un detector temprano de corpus drift.

Las **señales implícitas de calidad** del comportamiento de los usuarios son indicadores de satisfacción más directos que los scores del LLM-judge. Las más informativas son: queries seguidas de reformulación en menos de 30 segundos (señal de que la respuesta no fue satisfactoria), sesiones que terminan sin ninguna interacción positiva posterior (posible respuesta inaceptable), clic en las fuentes citadas (señal positiva de utilidad del sistema de atribución de fuentes), y queries repetidas del mismo usuario en la misma sesión (señal de que la respuesta no fue completa). Estas señales no son perfectas como métricas de calidad (un usuario puede reformular por precisar la query, no porque la respuesta fuera incorrecta), pero en agregado son proxies valiosos de la experiencia real del usuario.

Componentes del monitoring de calidad en producción

- **Distributed tracing con OpenTelemetry:** spans para query_embedding, vector_search, reranking, context_assembly, llm_generation; exportar a Langfuse, Phoenix (Arize AI) o Datadog APM; latencia por etapa y trazabilidad de fuentes por cada solicitud.
- **Evaluación asíncrona con LLM-as-judge:** 1–5% de las solicitudes de producción evaluadas asincrónicamente por faithfulness y answer relevancy; scores agregados

diariamente; alerta cuando la media diaria cae >10 puntos vs. baseline; overhead de evaluación nulo sobre la latencia de producción.

- **Embedding drift detection:** distribución diaria de scores de similitud del top-1 chunk; alerta cuando la media cae >5 puntos respecto al baseline; señal temprana de corpus drift o de distribución shift en queries.
- **Feedback implícito como proxy de calidad:** monitorear reformulaciones en <30 segundos, sesiones sin interacción positiva, clics en fuentes; agregar por ventana diaria y semanal; correlacionar con los scores del LLM-judge para calibrar su confiabilidad.
- **SLOs de calidad en producción:** definir faithfulness >0.75 , answer relevancy >0.70 como SLOs de calidad (no solo de latencia y disponibilidad); configurar alertas cuando los valores caen por debajo del umbral durante >24 horas; integrar con sistemas de on-call para degradaciones severas.
- **Herramientas de observabilidad LLM:** Langfuse (open source, tracing + evaluación + playground), LangSmith (nativo para LangChain, propietario), Phoenix de Arize AI (especializado en LLM observability con soporte explícito de spans RAG), TruLens (framework de evaluación con trazas).

La evaluación continua en producción cierra el ciclo de mejora continua del sistema: las métricas de producción revelan dónde el sistema falla en condiciones reales, esos fallos se incorporan como nuevos casos al golden dataset, y los cambios del pipeline se validan contra el golden dataset actualizado antes de desplegarse. Sin este ciclo, el sistema se degrada con el tiempo. Con él, el sistema mejora de forma sistemática. La siguiente sección cierra el capítulo con la reflexión sobre por qué la evaluación sistemática es la práctica que más distingue los equipos de AI Engineering de alto rendimiento.

Módulo 6 – Capítulo 06 – Sección 06

Cierre: sin evaluación sistemática, la mejora de RAG es aleatoria

Los equipos que construyen sistemas RAG sin una infraestructura de evaluación sólida terminan inevitablemente en el mismo lugar: un ciclo de optimización ciega donde los cambios al pipeline se basan en intuición, en la experiencia de casos individuales anecdóticos, o en la recomendación del último paper que el equipo leyó. En este contexto, es imposible determinar si un cambio ha mejorado o degradado el sistema de forma sistemática: la calidad puede fluctuar por razones completamente ajenas al cambio —una actualización silenciosa del modelo de API, un cambio en la distribución de queries, un documento corrupto que se coló en el corpus— y el equipo no tiene forma de distinguir el efecto del cambio del ruido de fondo. Las "mejoras" pueden ser regresiones disfrazadas de progreso, y los problemas reales permanecen sin diagnóstico.

La evaluación sistemática convierte el desarrollo de RAG en un proceso de ingeniería reproducible con las mismas propiedades que cualquier otro proceso de ingeniería de calidad: cada cambio tiene una hipótesis medible ("este nuevo chunking debería mejorar Recall@5 en queries de síntesis en al menos 5 puntos"), una métrica que confirma o refuta la hipótesis, y un criterio objetivo de aceptación o rechazo del cambio. Los equipos más avanzados en el campo —los de Cohere, Databricks y Elastic que han publicado post-mortems técnicos sobre sus sistemas RAG— reportan consistentemente que la inversión en infraestructura de evaluación retorna en velocidad de iteración: cada experimento tarda horas en lugar de días porque el pipeline de evaluación automatizado produce resultados en minutos, y cada cambio llega a producción con evidencia cuantitativa de que mejora el sistema.

La infraestructura de evaluación que este capítulo describió tiene tres niveles: el golden dataset como referencia estática de calidad (100–500 queries anotadas por expertos del dominio, cubierto en la Sección 4); el pipeline de evaluación automatizado con RAGAS que corre en CI/CD para cada cambio del pipeline (descrito en las Secciones 2 y 3); y el monitoring continuo de producción con LLM-as-judge y señales implícitas de usuario (descrito en la Sección 5). Los tres niveles son complementarios: el golden dataset

proporciona la referencia estable de calidad, el pipeline CI/CD detecta regresiones antes del despliegue, y el monitoring de producción detecta degradaciones que no eran visibles en el dataset estático.

Esta infraestructura no es costosa de construir: el golden dataset inicial de 100 queries requiere 1–2 días de trabajo experto; la integración de RAGAS en el pipeline CI/CD es menos de una tarde de trabajo técnico; Langfuse es open source y puede desplegarse en horas. El costo de no tenerla es inmensamente mayor: semanas de diagnóstico manual de problemas de calidad, deployments que degradan la experiencia del usuario sin que el equipo lo detecte a tiempo, y un sistema cuya calidad es impredecible porque nadie la mide. La evaluación no es un overhead del proceso de desarrollo de RAG; es la práctica que distingue la ingeniería de sistemas RAG de la experimentación no controlada.

El Capítulo 7 construye sobre esta base de evaluación para abordar el problema de operar el sistema RAG completo en producción: cómo desacoplar la arquitectura para escalar, cómo gestionar pipelines de ingesta a escala, cómo versionar índices y modelos, y cómo implementar la observabilidad y seguridad que los entornos empresariales exigen.

"Midiendo lo que importa: sin datos cuantitativos sobre el rendimiento de un sistema, toda mejora es una hipótesis no verificada." — principio de gestión basada en datos adaptado al contexto de la ingeniería de calidad en sistemas de IA, parafraseando a John Doerr en su aplicación del marco OKR a métricas medibles.